Name: Monisha Raghini Chelle
RollNo:2520030537

Skill – 1

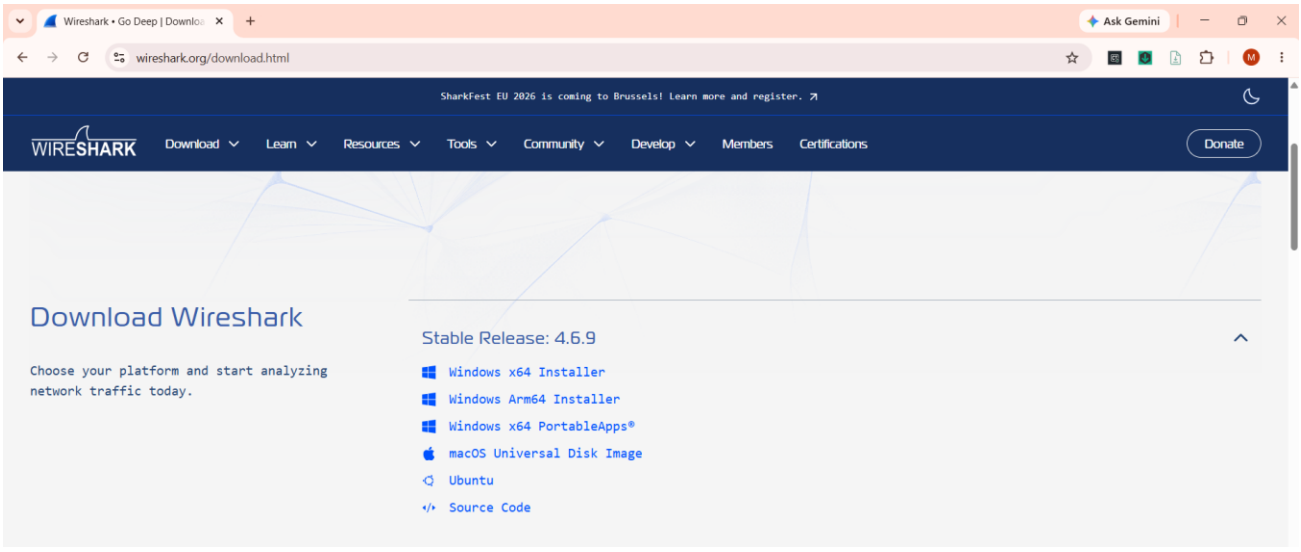
Encapsulation Builder: Analyze the overhead introduced by protocol headers and determine how encapsulation affects network efficiency.

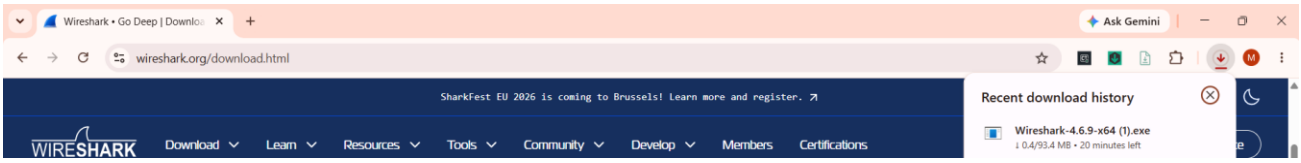
Aim: To capture network packets and understand how Ethernet, IP, and TCP/UDP headers are added during data transmission.

Whenever data travels through a computer network, each protocol layer adds its own header to the data. These headers introduce overhead. Using Wireshark, we will capture real network packets, examine Ethernet, IP and TCP/UDP headers, calculate the total overhead, and determine how efficiently the network carries application data.

Step 1: Install Wireshark

1. Open the browser.
2. Install Wireshark [Wireshark • Go Deep | Download]
3. Install it using the default settings.
4. Open Wireshark.



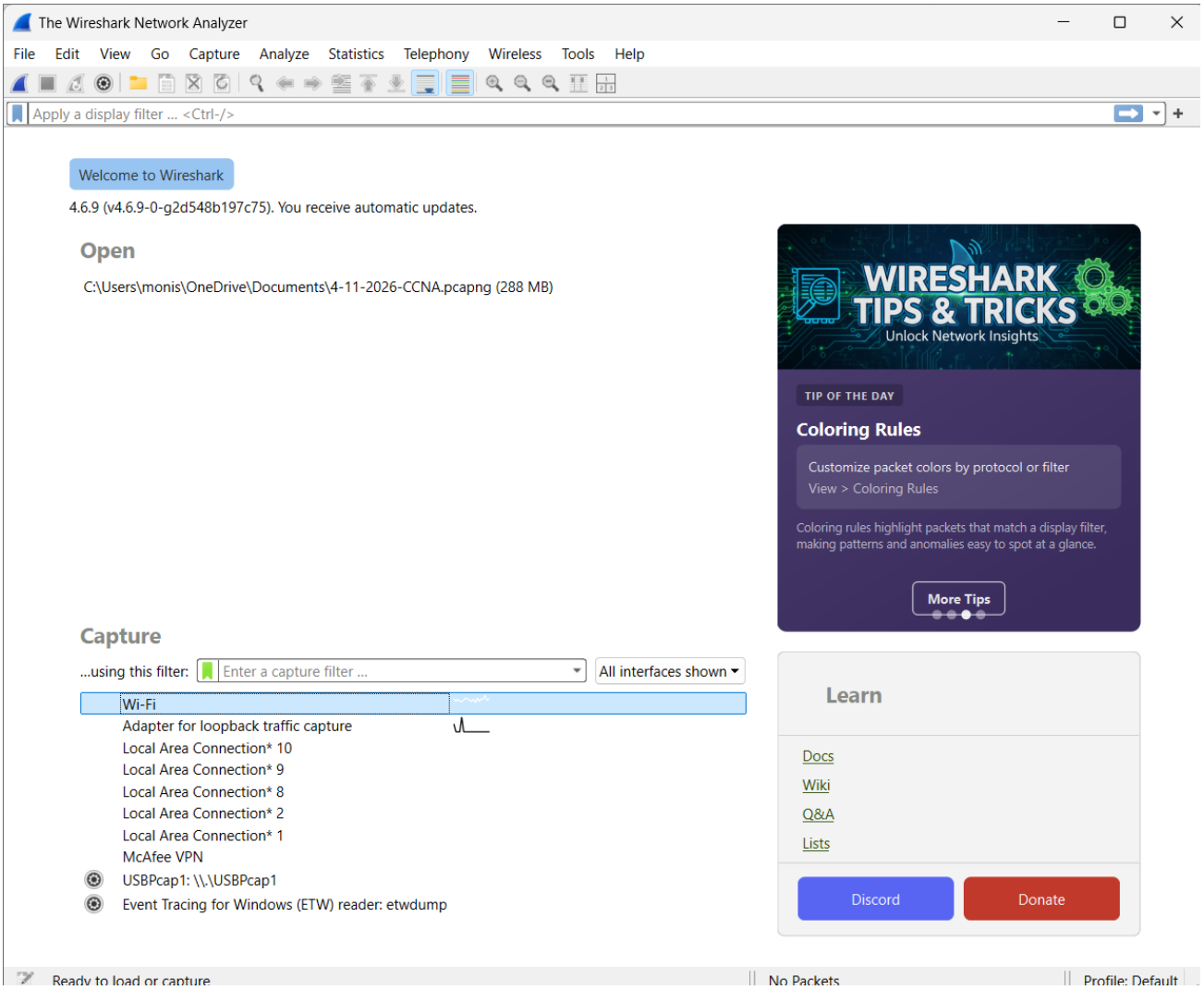


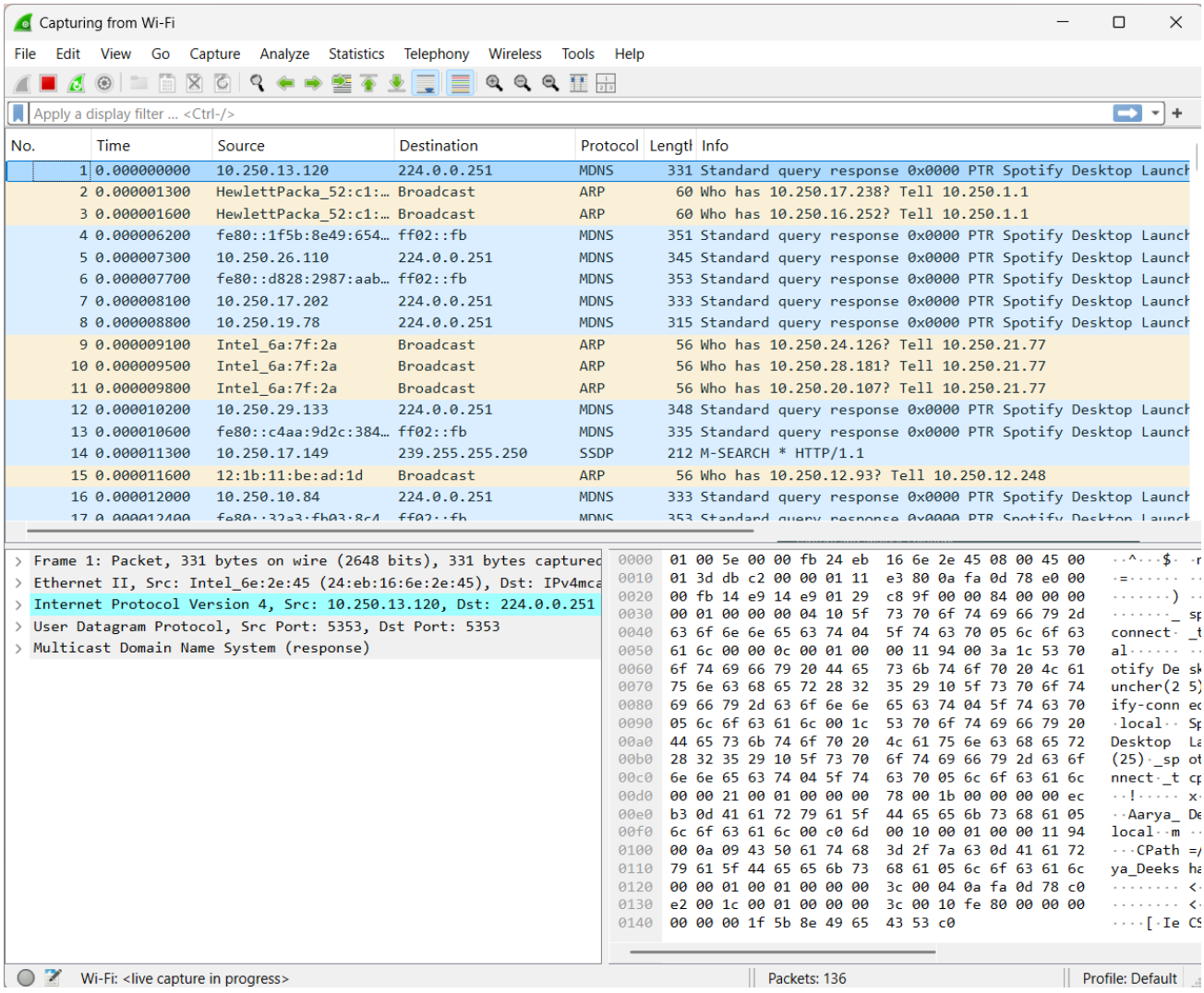
Step 2: Select your network interface

When Wireshark opens, you will see interfaces such as:

Wi-Fi, Ethernet, Bluetooth

1. If you are connected through Wi-Fi, select Wi-Fi.
2. Double-click the Wi-Fi interface.
3. Packet capture will start.


The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Welcome to Wireshark
4.6.9 (v4.6.9-0-g2d548b197c75). You receive automatic updates.
Open
C:\Users\monis\OneDrive\Documents\4-11-2026-CCNA.pcapng (288 MB)
WIRESHARK TIPS & TRICKS
Unlock Network Insights
TIP OF THE DAY
Coloring Rules
Customize packet colors by protocol or filter
View > Coloring Rules
Coloring rules highlight packets that match a display filter, making patterns and anomalies easy to spot at a glance.
More Tips
Capture
...using this filter: Enter a capture filter ...
All interfaces shown
Wi-Fi
Adapter for loopback traffic capture
Local Area Connection* 10
Local Area Connection* 9
Local Area Connection* 8
Local Area Connection* 2
Local Area Connection* 1
McAfee VPN
USBPcap1: \\.\USBPcap1
Event Tracing for Windows (ETW) reader: etwdump
Learn
Docs
Wiki
Q&A
Lists
Discord
Donate
Ready to load or capture
No Packets
Profile: Default



Step 3– Generate network traffic

Keep Wireshark capturing. Do not stop it yet.

Open Google Chrome and visit:

1. https://www.google.com
2. https://www.youtube.com
3. https://www.wikipedia.org

Now generate DNS traffic.

1. Press Windows + R.
2. Type cmd.
3. Press Enter.
4. Execute this command:

nslookup google.com

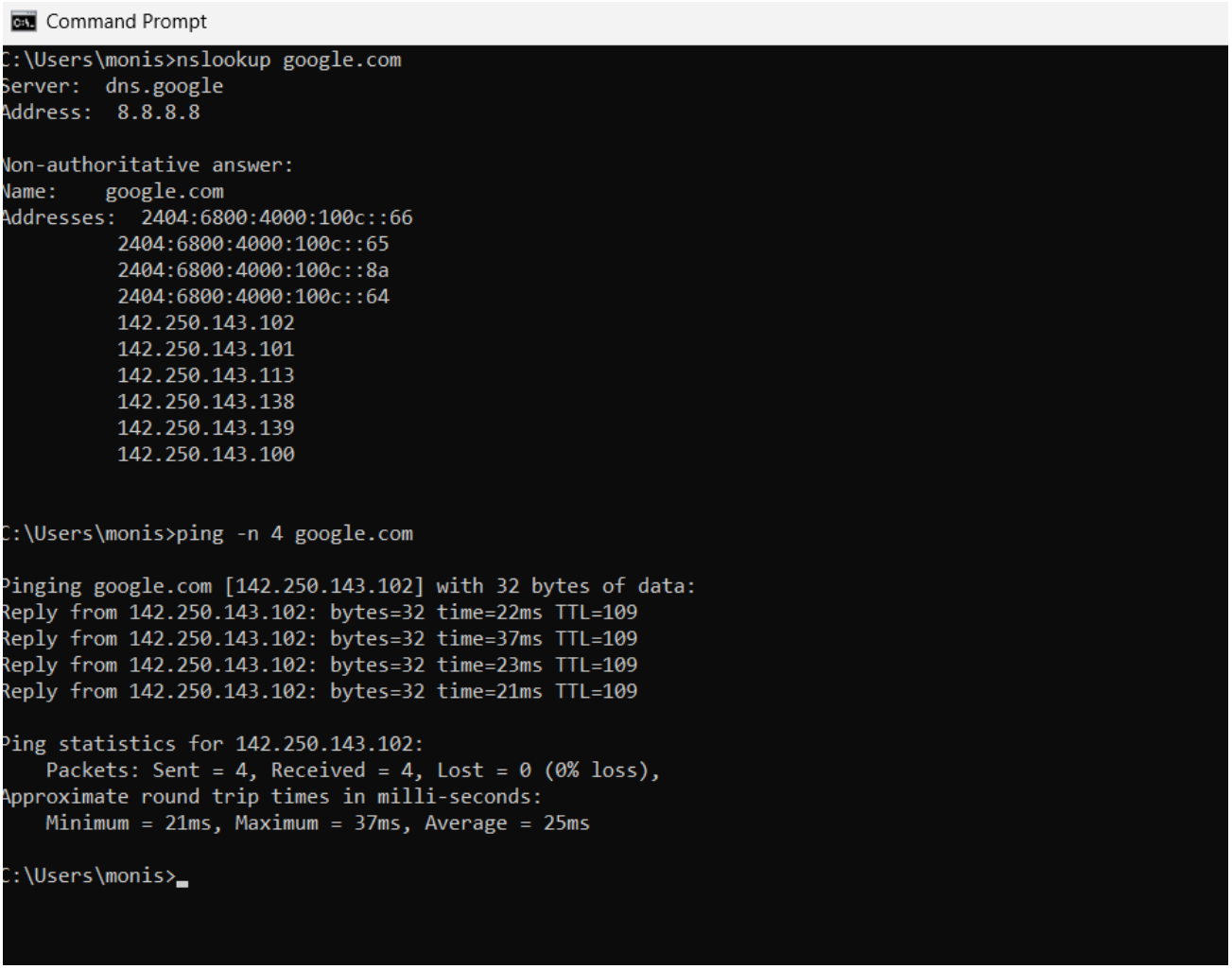
5. Press Enter.

This generates a DNS lookup, which normally uses UDP.

Next, execute:

ping -n 4 google.com

This generates ICMP traffic if your network permits ping.



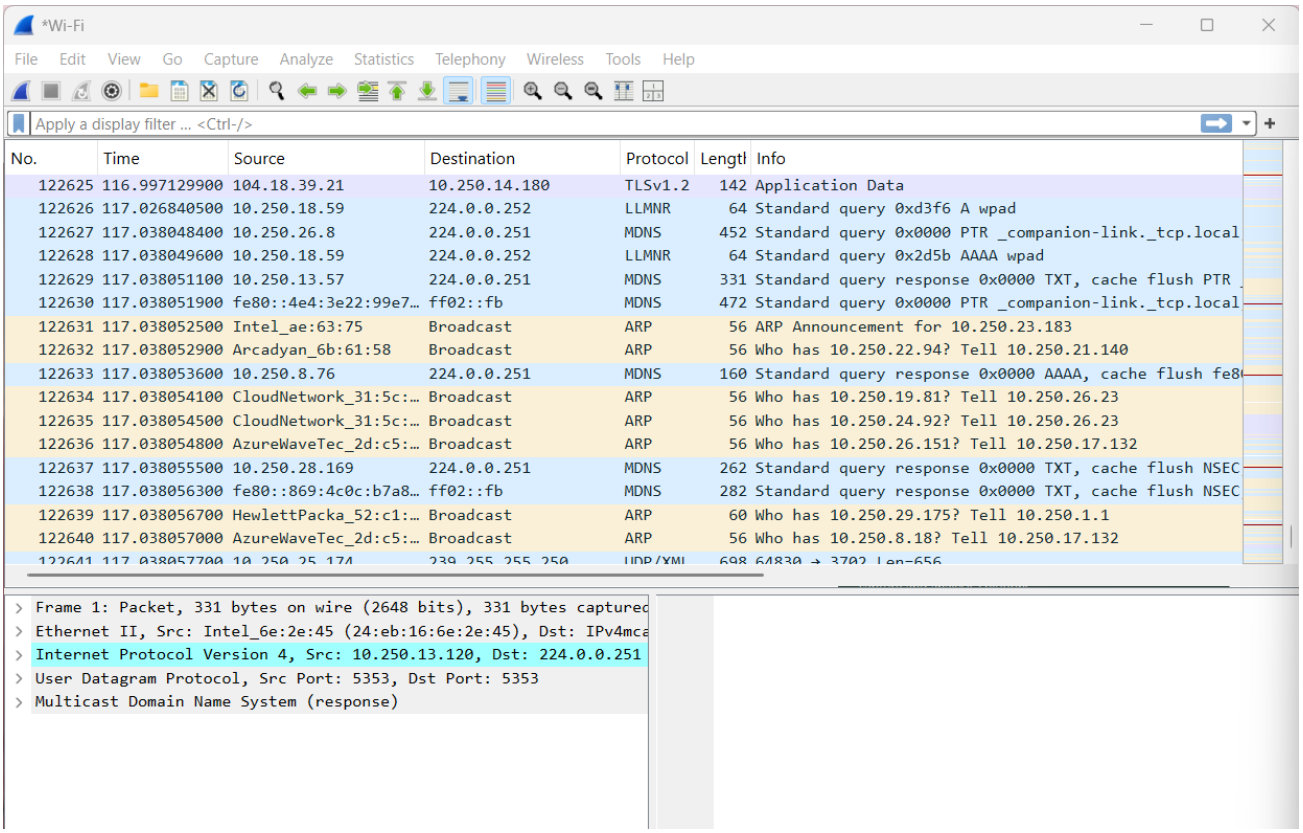
Wait around 30 seconds.

While Wireshark is capturing:

1. Open a new browser tab.
2. Visit a website such as https://www.youtube.com
3. Open two or three different websites.
4. Wait for approximately 30 seconds.
5. Return to Wireshark.

6.Click the red Stop button.

You should now see captured packets.



Step 4: Apply display filters

In the Wireshark filter bar, enter each filter separately and press Enter.

Filter	**Purpose**
tcp	Displays TCP packets
udp	Displays UDP packets

icmp	Displays ICMP packets
tcp.len > 0	Displays TCP packets carrying data

Apply the four display filters

Now we'll use the same captured packets for all four filters.

The filter bar is at the top of Wireshark.

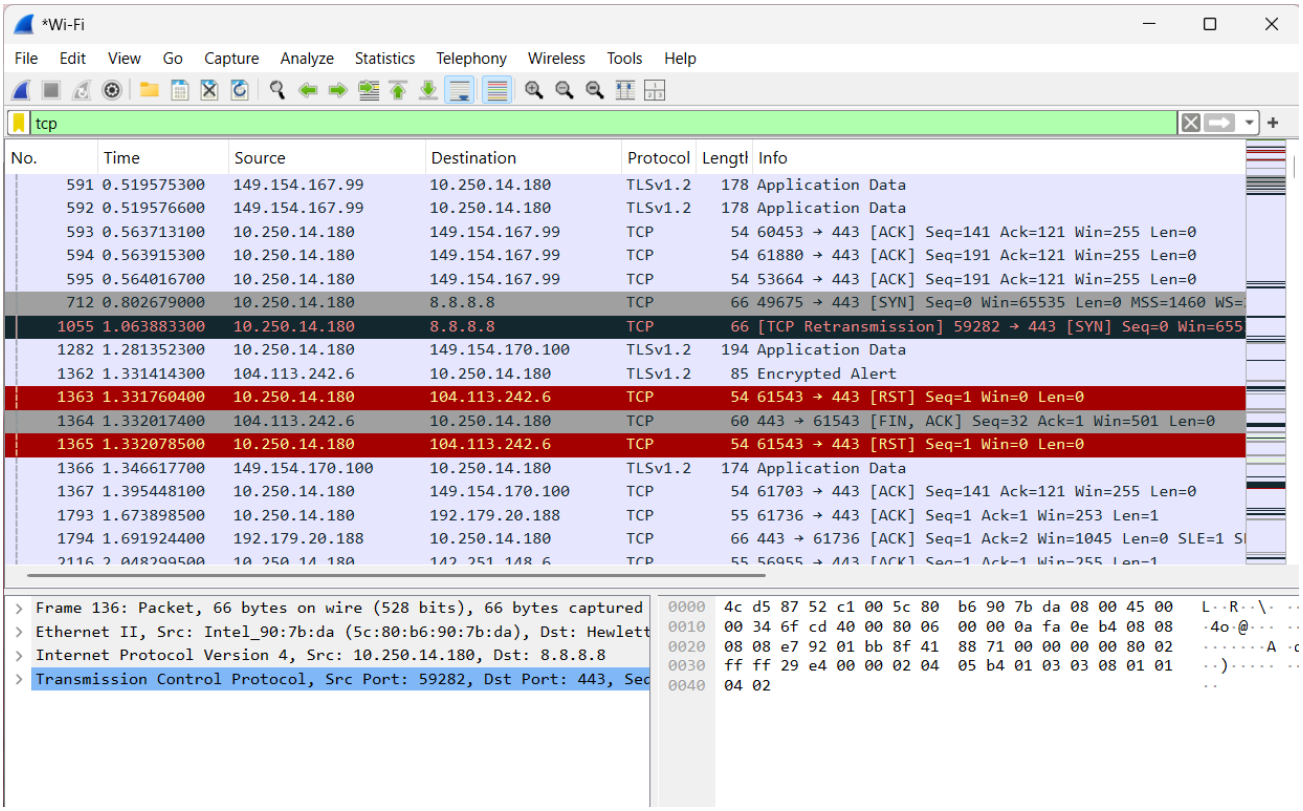
1. TCP filter

Enter:

tcp

Press Enter.

You should see TCP packets. Some may be labelled TLSv1.2 or TLSv1.3 because HTTPS uses TLS over TCP.



2.UDP filter

Enter:

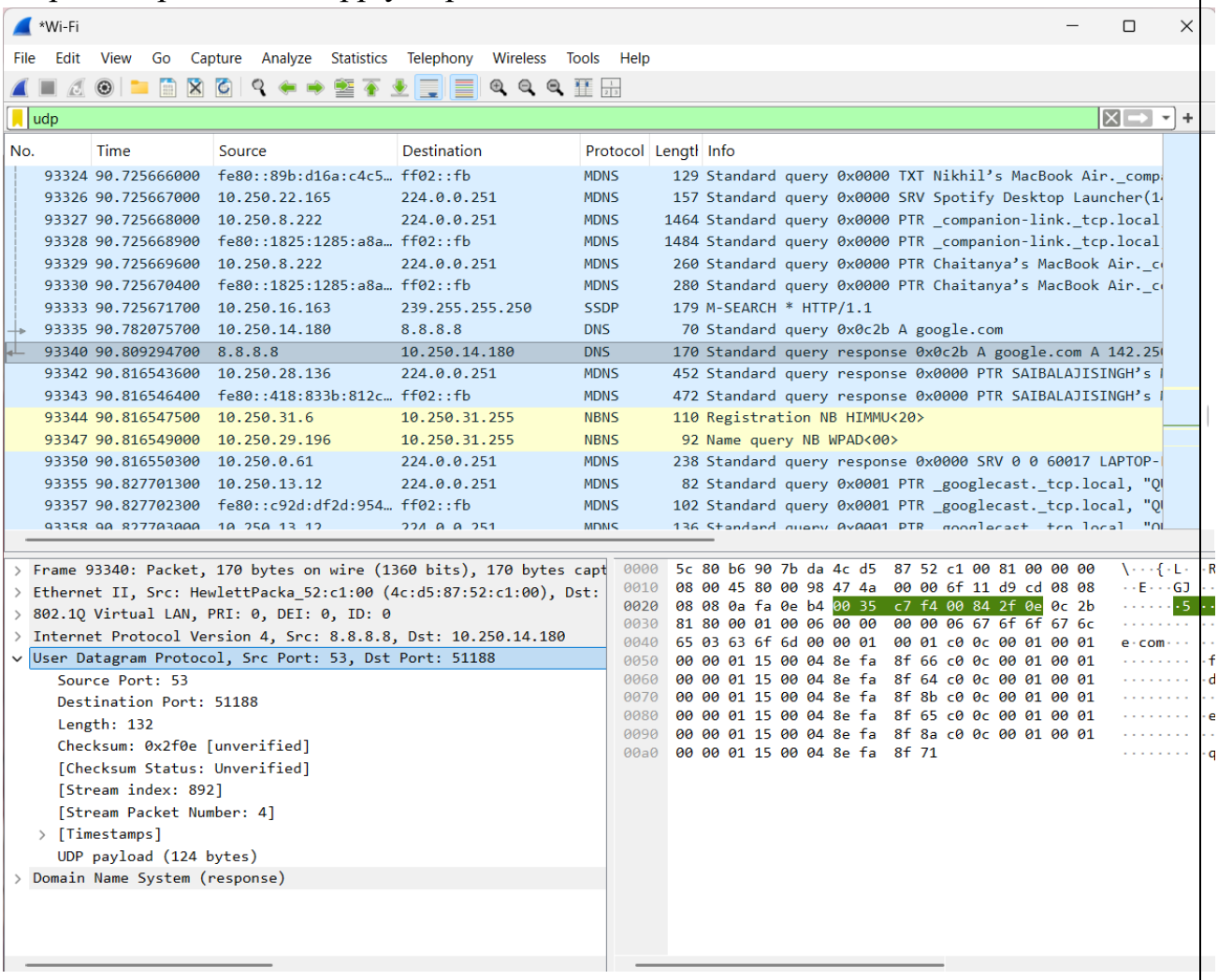
udp

Press Enter.

You should see UDP packets. Look for DNS traffic in the Info column.

If you don't see UDP:

- Check that you generated DNS traffic while Wireshark was capturing.
- Clear the filter.
- Start a fresh capture.
- Run nslookup google.com again.
- Stop the capture and reapply udp.



3. ICMP filter

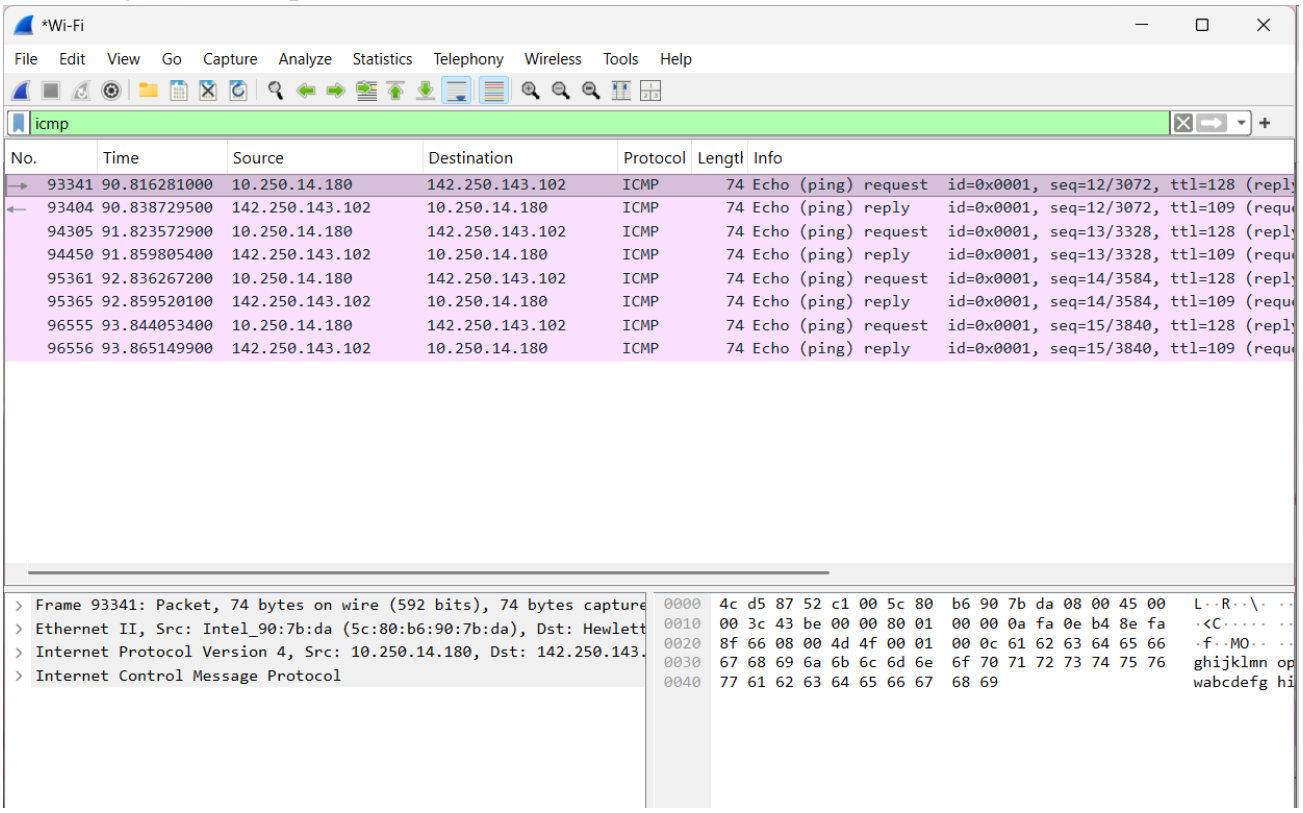
Enter:

icmp

Press Enter.

You should see ICMP packets if your ping generated responses or requests.

If ICMP is missing, ping may be blocked by your network. Try ping -n 4 8.8.8.8 during a fresh capture.



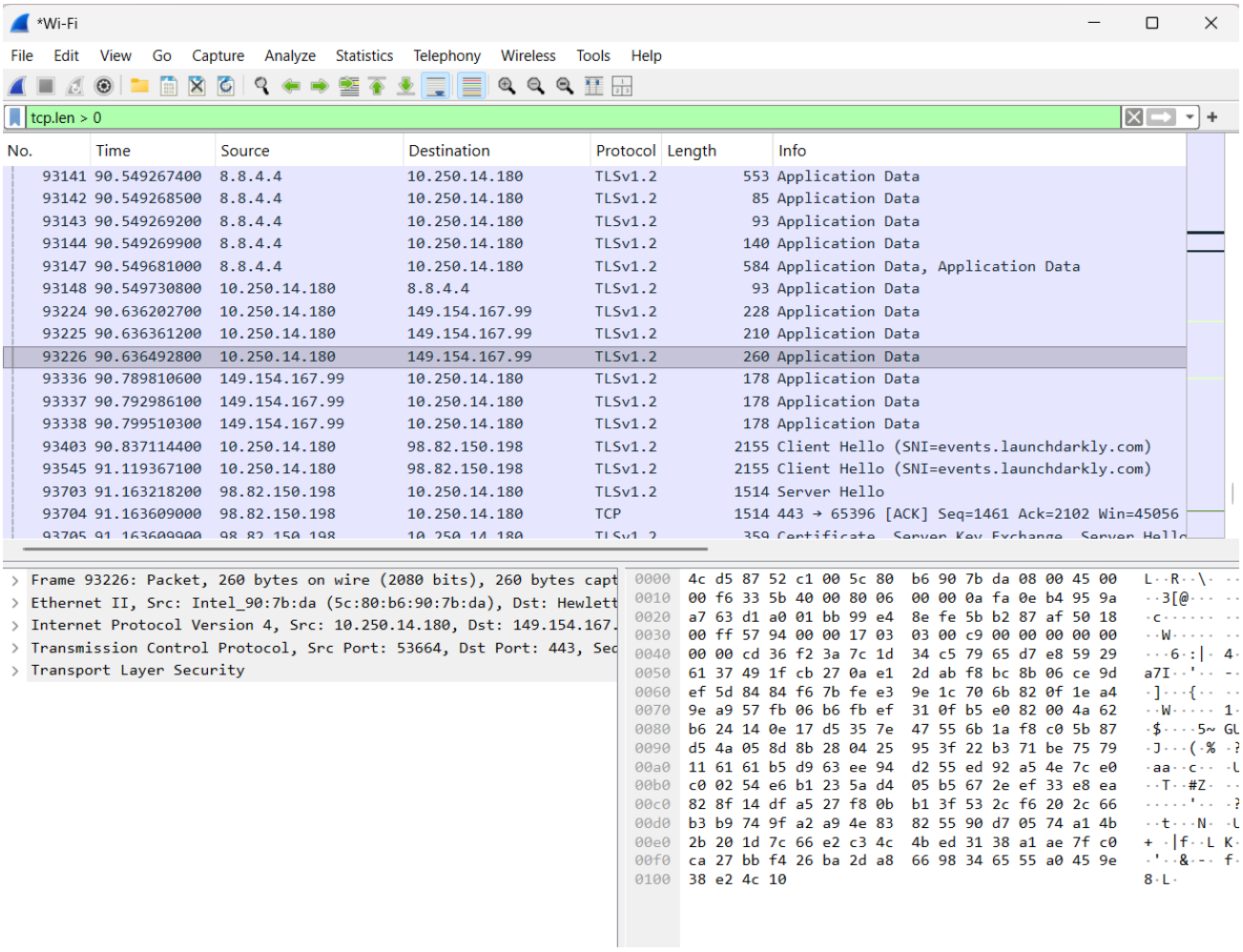
4.TCP packets carrying data

Enter:

tcp.len > 0

Press Enter.

This displays TCP packets with a non-zero TCP payload.

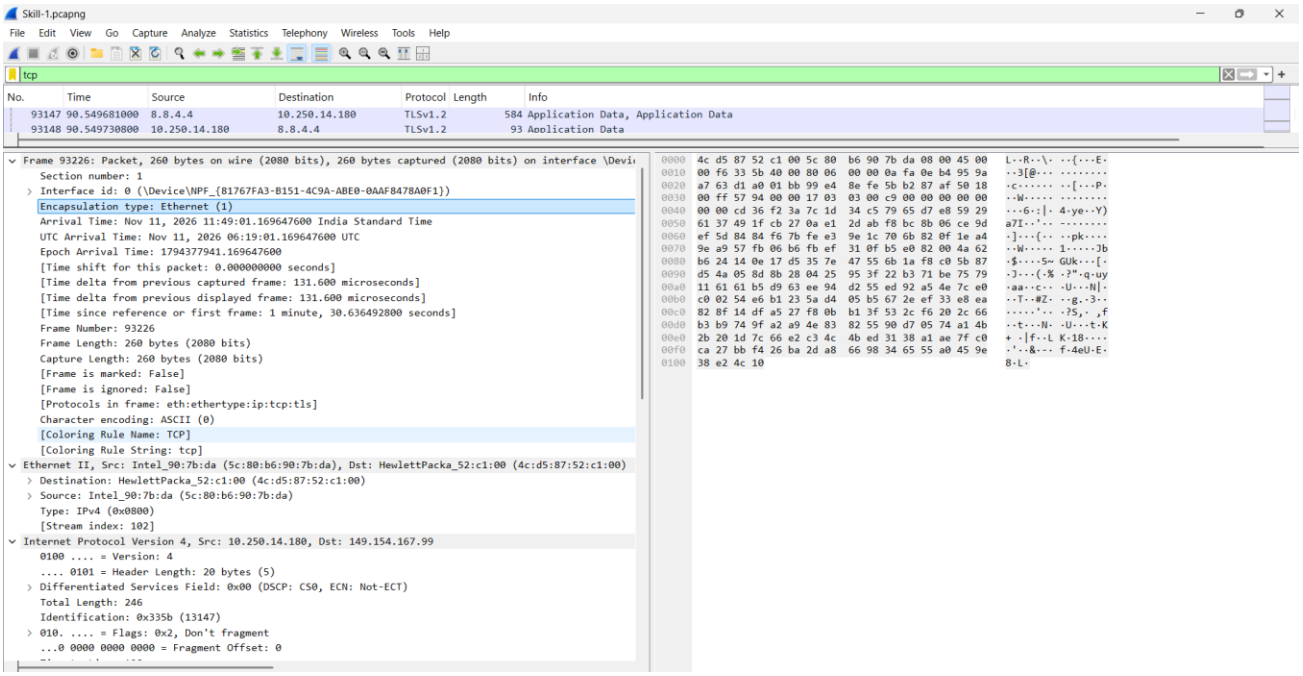


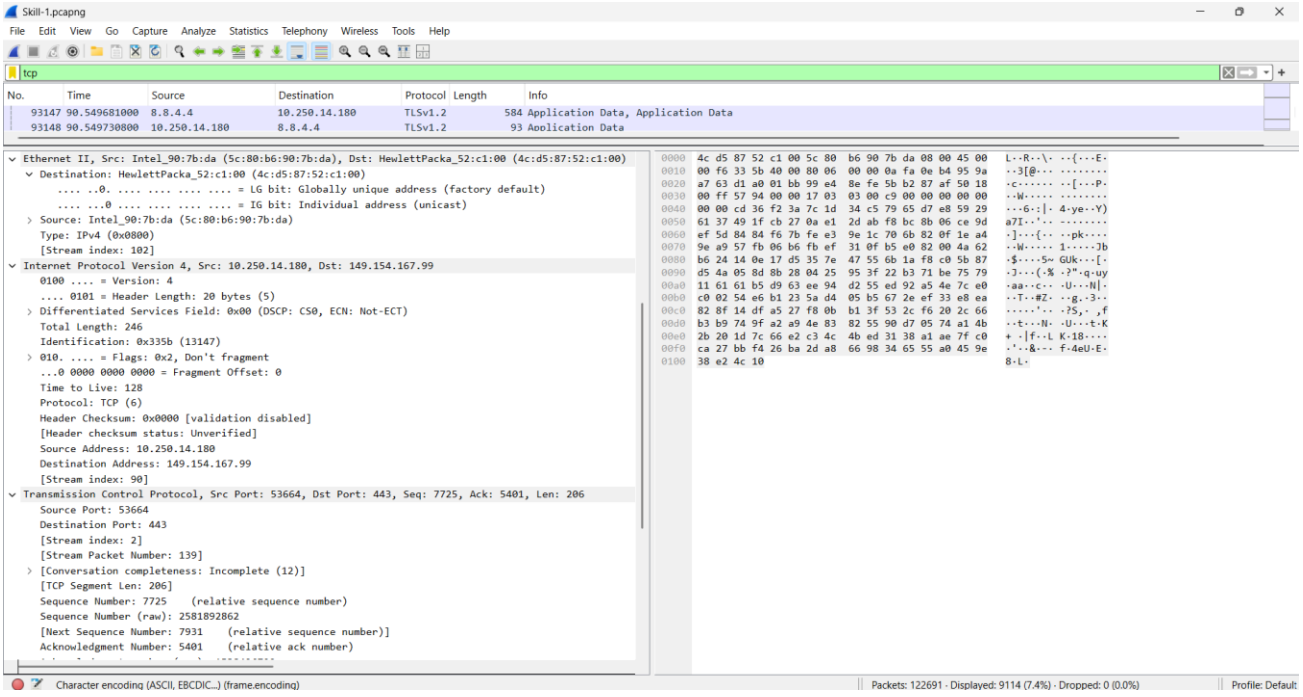
Step 5: Inspect packet headers

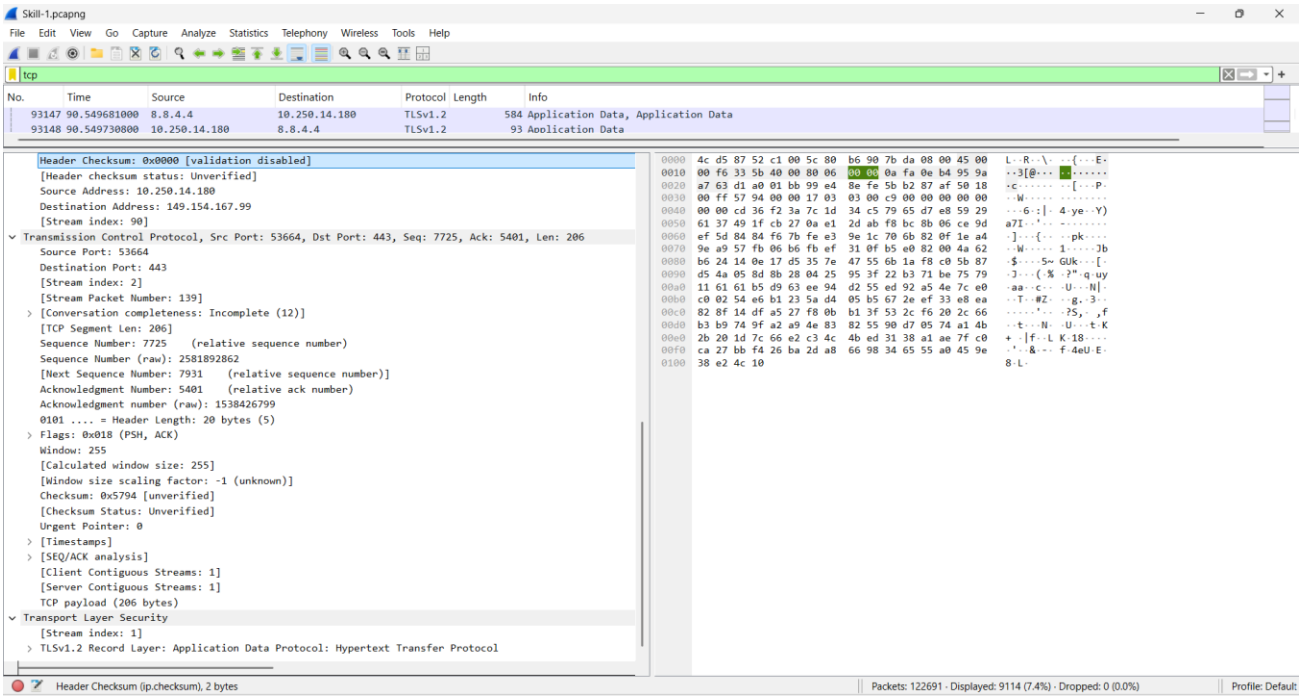
Select any **TCP** packet from the packet list.

In the packet details pane, expand these sections:

1. Frame
2. Ethernet II
3. Internet Protocol Version 4
4. Transmission Control Protocol


Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp
No. Time Source Destination Protocol Length Info
93147 90.549681000 8.8.4.4 10.250.14.180 TLSv1.2 584 Application Data, Application Data
93148 90.549730800 10.250.14.180 8.8.4.4 TLSv1.2 93 Application Data
Frame 93226: Packet, 260 bytes on wire (2080 bits), 260 bytes captured (2080 bits) on interface \Devi
Section number: 1
Interface id: 0 (\Device\NPF_{81767FA3-B151-4C9A-ABE0-0AAF8478A0F1})
Encapsulation type: Ethernet (1)
Arrival Time: Nov 11, 2026 11:49:01.169647600 India Standard Time
UTC Arrival Time: Nov 11, 2026 06:19:01.169647600 UTC
Epoch Arrival Time: 1794377941.169647600
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 131.600 microseconds]
[Time delta from previous displayed frame: 131.600 microseconds]
[Time since reference or first frame: 1 minute, 30.636492800 seconds]
Frame Number: 93226
Frame Length: 260 bytes (2080 bits)
Capture Length: 260 bytes (2080 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp:tls]
Character encoding: ASCII (0)
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]
Ethernet II, Src: Intel_90:7b:da (5c:80:b6:90:7b:da), Dst: HewlettPacka_52:c1:00 (4c:d5:87:52:c1:00)
Destination: HewlettPacka_52:c1:00 (4c:d5:87:52:c1:00)
Source: Intel_90:7b:da (5c:80:b6:90:7b:da)
Type: IPv4 (0x0800)
[Stream index: 102]
Internet Protocol Version 4, Src: 10.250.14.180, Dst: 149.154.167.99
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 246
Identification: 0x335b (13147)
010. .... = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
0000 4c d5 87 52 c1 00 5c 80 b6 90 7b da 08 00 45 00
0010 00 f6 33 5b 40 00 80 06 00 00 0a fa 0e b4 95 9a
0020 a7 63 d1 a0 01 bb 99 e4 8e fe 5b b2 87 af 50 18
0030 00 ff 57 94 00 00 17 03 03 00 c9 00 00 00 00 00
0040 00 00 cd 36 f2 3a 7c 1d 34 c5 79 65 d7 e8 59 29
0050 61 37 49 1f cb 27 0a e1 2d ab f8 bc 8b 06 ce 9d
0060 ef 5d 84 84 f6 7b fe e3 9e 1c 70 6b 82 0f 1e a4
0070 9e a9 57 fb 06 b6 fb ef 31 0f b5 e0 82 00 4a 62
0080 b6 24 14 0e 17 d5 35 7e 47 55 6b 1a f8 c0 5b 87
0090 d5 4a 05 8d 8b 28 04 25 95 3f 22 b3 71 be 75 79
00a0 11 61 61 b5 d9 63 ee 94 d2 55 ed 92 a5 4e 7c e0
00b0 c0 02 54 e6 b1 23 5a d4 05 b5 67 2e ef 33 e8 ea
00c0 82 8f 14 df a5 27 f8 0b b1 3f 53 2c f6 20 2c 66
00d0 b3 b9 74 9f a2 a9 4e 83 82 55 90 d7 05 74 a1 4b
00e0 2b 20 1d 7c 66 e2 c3 4c 4b ed 31 38 a1 ae 7f c0
00f0 ca 27 bb f4 26 ba 2d a8 66 98 34 65 55 a0 45 9e
0100 38 e2 4c 10

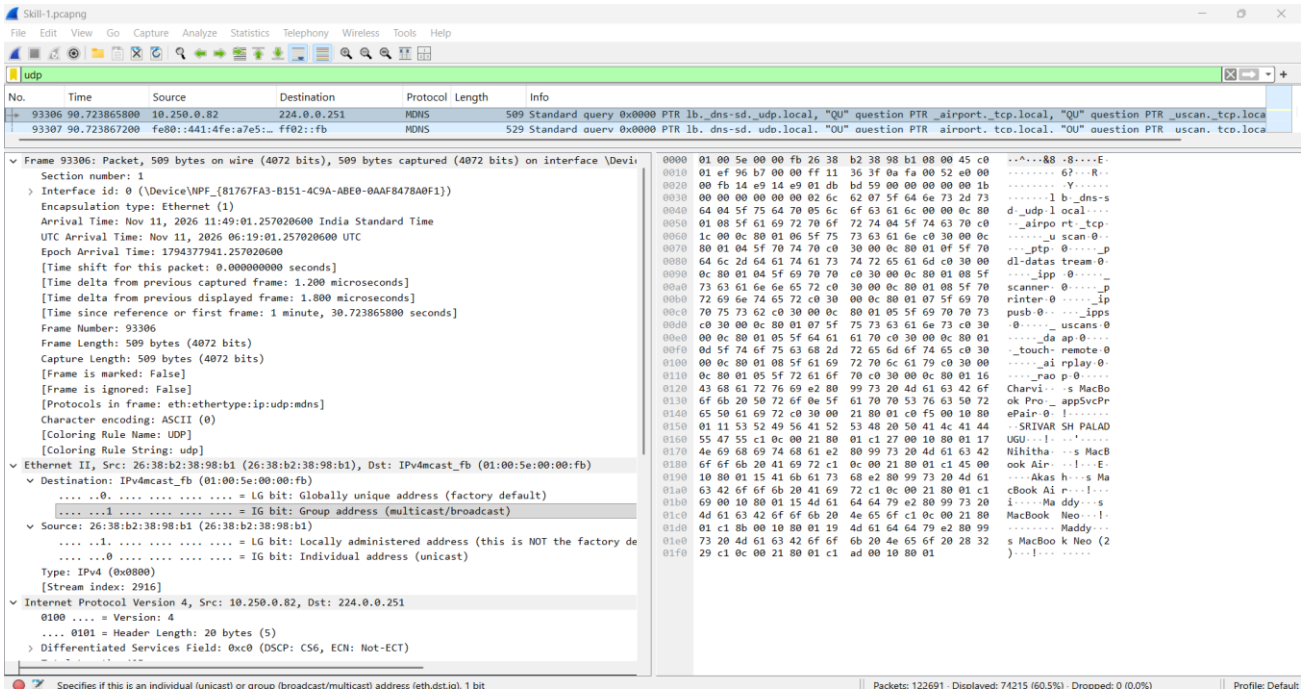

Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp
No. Time Source Destination Protocol Length Info
93147 90.549681000 8.8.4.4 10.250.14.180 TLSv1.2 584 Application Data, Application Data
93148 90.549730800 10.250.14.180 8.8.4.4 TLSv1.2 93 Application Data
Ethernet II, Src: Intel_90:7b:da (5c:80:b6:90:7b:da), Dst: HewlettPacka_52:c1:00 (4c:d5:87:52:c1:00)
Destination: HewlettPacka_52:c1:00 (4c:d5:87:52:c1:00)
.... ..0. .... = LG bit: Globally unique address (factory default)
....0 .... = IG bit: Individual address (unicast)
Source: Intel_90:7b:da (5c:80:b6:90:7b:da)
Type: IPv4 (0x0800)
[Stream index: 102]
Internet Protocol Version 4, Src: 10.250.14.180, Dst: 149.154.167.99
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 246
Identification: 0x335b (13147)
010. .... = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: TCP (6)
Header Checksum: 0x0000 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.250.14.180
Destination Address: 149.154.167.99
[Stream index: 90]
Transmission Control Protocol, Src Port: 53664, Dst Port: 443, Seq: 7725, Ack: 5401, Len: 206
Source Port: 53664
Destination Port: 443
[Stream index: 2]
[Stream Packet Number: 139]
[Conversation completeness: Incomplete (12)]
[TCP Segment Len: 206]
Sequence Number: 7725 (relative sequence number)
Sequence Number (raw): 2581892862
[Next Sequence Number: 7931 (relative sequence number)]
Acknowledgment Number: 5401 (relative ack number)
0000 4c d5 87 52 c1 00 5c 80 b6 90 7b da 08 00 45 00
0010 00 f6 33 5b 40 00 80 06 00 00 0a fa 0e b4 95 9a
0020 a7 63 d1 a0 01 bb 99 e4 8e fe 5b b2 87 af 50 18
0030 00 ff 57 94 00 00 17 03 03 00 c9 00 00 00 00 00
0040 00 00 cd 36 f2 3a 7c 1d 34 c5 79 65 d7 e8 59 29
0050 61 37 49 1f cb 27 0a e1 2d ab f8 bc 8b 06 ce 9d
0060 ef 5d 84 84 f6 7b fe e3 9e 1c 70 6b 82 0f 1e a4
0070 9e a9 57 fb 06 b6 fb ef 31 0f b5 e0 82 00 4a 62
0080 b6 24 14 0e 17 d5 35 7e 47 55 6b 1a f8 c0 5b 87
0090 d5 4a 05 8d 8b 28 04 25 95 3f 22 b3 71 be 75 79
00a0 11 61 61 b5 d9 63 ee 94 d2 55 ed 92 a5 4e 7c e0
00b0 c0 02 54 e6 b1 23 5a d4 05 b5 67 2e ef 33 e8 ea
00c0 82 8f 14 df a5 27 f8 0b b1 3f 53 2c f6 20 2c 66
00d0 b3 b9 74 9f a2 a9 4e 83 82 55 90 d7 05 74 a1 4b
00e0 2b 20 1d 7c 66 e2 c3 4c 4b ed 31 38 a1 ae 7f c0
00f0 ca 27 bb f4 26 ba 2d a8 66 98 34 65 55 a0 45 9e
0100 38 e2 4c 10
Character encoding (ASCII, EBCDIC...) (frame.encoding)
Packets: 122691 · Displayed: 9114 (7.4%) · Dropped: 0 (0.0%)
Profile: Default

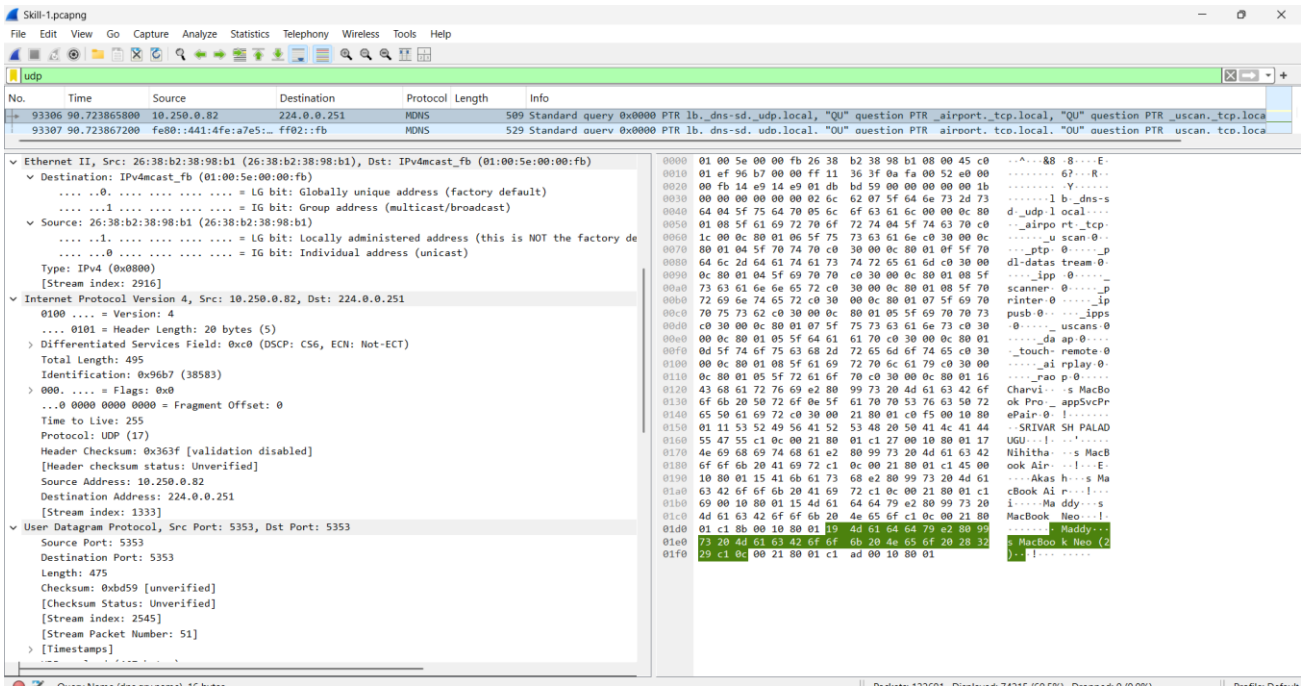


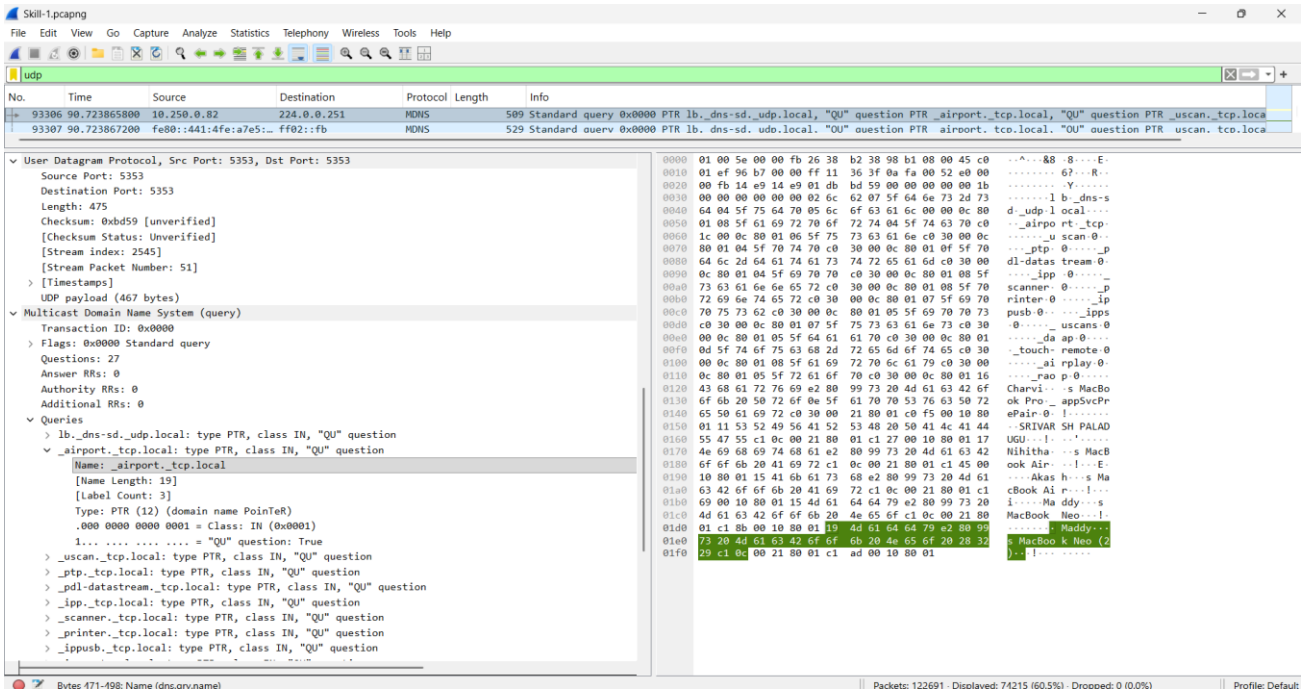
Select any **UDP** packet from the packet list.

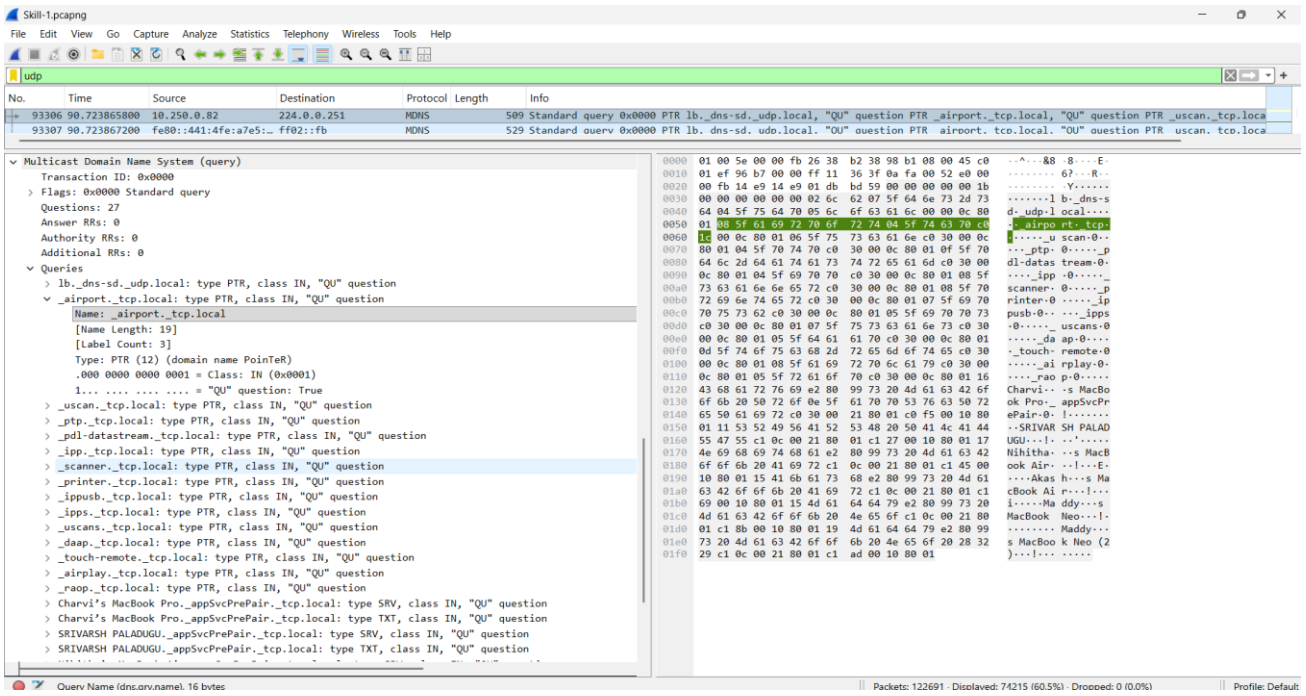
In the packet details pane, expand these sections:

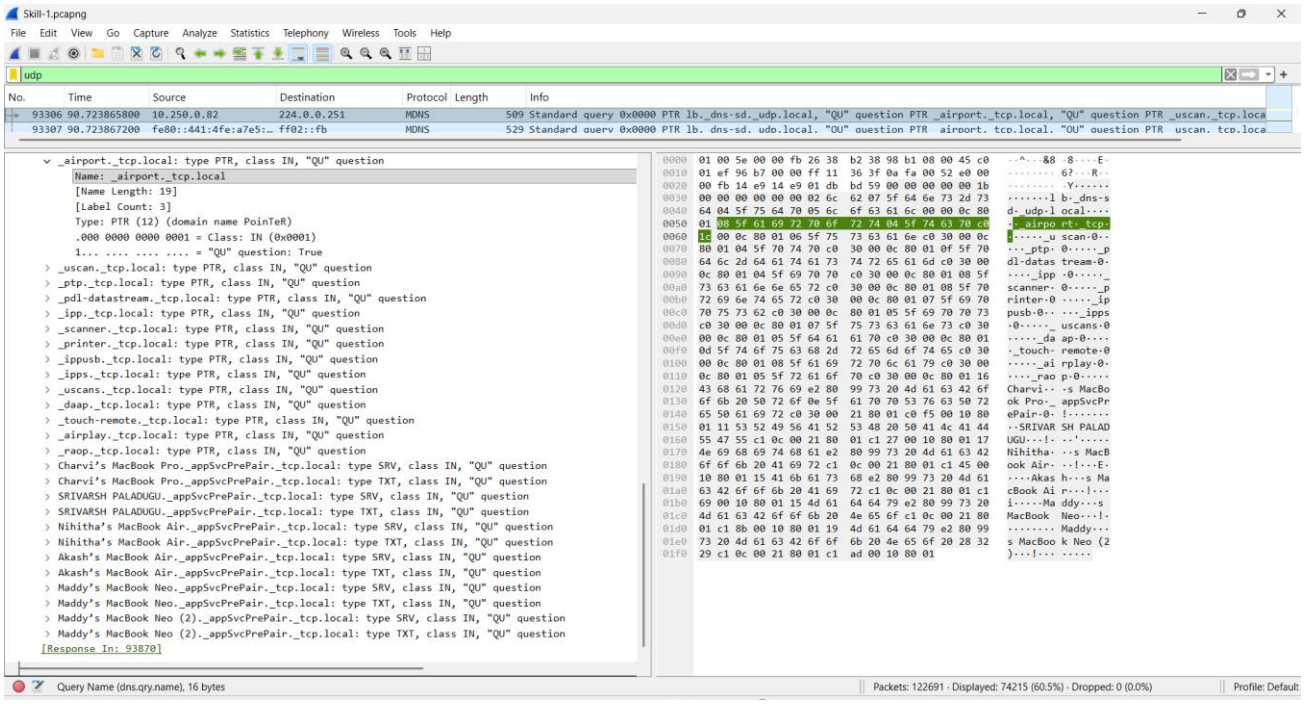
5. Frame
6. Ethernet II
7. Internet Protocol Version 4
8. User Datagram Protocol


Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Length Info
93306 90.723865800 10.250.0.82 224.0.0.251 MDNS 509 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
93307 90.723867200 fe80::441:4fe:a7e5:… ff02::fb MDNS 529 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
Frame 93306: Packet, 509 bytes on wire (4072 bits), 509 bytes captured (4072 bits) on interface \Devi
Section number: 1
Interface id: 0 (\Device\NPF_{81767FA3-B151-4C9A-ABE0-0AAF8478A0F1})
Encapsulation type: Ethernet (1)
Arrival Time: Nov 11, 2026 11:49:01.257020600 India Standard Time
UTC Arrival Time: Nov 11, 2026 06:19:01.257020600 UTC
Epoch Arrival Time: 1794377941.257020600
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 1.200 microseconds]
[Time delta from previous displayed frame: 1.800 microseconds]
[Time since reference or first frame: 1 minute, 30.723865800 seconds]
Frame Number: 93306
Frame Length: 509 bytes (4072 bits)
Capture Length: 509 bytes (4072 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:udp:mdns]
Character encoding: ASCII (0)
[Coloring Rule Name: UDP]
[Coloring Rule String: udp]
Ethernet II, Src: 26:38:b2:38:98:b1 (26:38:b2:38:98:b1), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
Destination: IPv4mcast_fb (01:00:5e:00:00:fb)
.... ..0. .... = LG bit: Globally unique address (factory default)
....1 .... = IG bit: Group address (multicast/broadcast)
Source: 26:38:b2:38:98:b1 (26:38:b2:38:98:b1)
.... ..1. .... = LG bit: Locally administered address (this is NOT the factory de
....0 .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 2916]
Internet Protocol Version 4, Src: 10.250.0.82, Dst: 224.0.0.251
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
0000 01 00 5e 00 00 fb 26 38 b2 38 98 b1 08 00 45 c0
0010 01 ef 96 b7 00 00 ff 11 36 3f 0a fa 00 52 e0 00
0020 00 fb 14 e9 14 e9 01 db bd 59 00 00 00 00 00 1b
0030 00 00 00 00 00 00 02 6c 62 07 5f 64 6e 73 2d 73
0040 64 04 5f 75 64 70 05 6c 6f 63 61 6c 00 00 0c 80
0050 01 08 5f 61 69 72 70 6f 72 74 04 5f 74 63 70 c0
0060 1c 00 0c 80 01 06 5f 75 73 63 61 6e c0 30 00 0c
0070 80 01 04 5f 70 74 70 c0 30 00 0c 80 01 0f 5f 70
0080 64 6c 2d 64 61 74 61 73 74 72 65 61 6d c0 30 00
0090 0c 80 01 04 5f 69 70 70 c0 30 00 0c 80 01 08 5f
00a0 73 63 61 6e 6e 65 72 c0 30 00 0c 80 01 08 5f 70
00b0 72 69 6e 74 65 72 c0 30 00 0c 80 01 07 5f 69 70
00c0 70 75 73 62 c0 30 00 0c 80 01 05 5f 69 70 73 73
00d0 c0 30 00 0c 80 01 07 5f 75 73 63 61 6e 73 c0 30
00e0 00 0c 80 01 05 5f 64 61 70 c0 30 00 0c 80 01
00f0 0d 5f 74 6f 75 63 68 2d 72 65 6d 6f 74 65 c0 30
0100 00 0c 80 01 08 5f 61 69 72 70 6c 61 79 c0 30 00
0110 0c 80 01 05 5f 72 61 6f 70 c0 30 00 0c 80 01 16
0120 43 68 61 72 76 69 e2 80 99 73 20 4d 61 63 42 6f
0130 6f 6b 20 50 72 6f 0e 5f 61 70 70 53 76 63 50 72
0140 65 50 61 69 72 c0 30 00 21 80 01 c0 f5 00 10 80
0150 01 11 53 52 49 56 41 52 53 48 20 50 41 4c 41 44
0160 55 47 55 c1 0c 00 21 80 01 c1 27 00 10 80 01 17
0170 4e 69 68 69 74 68 61 e2 80 99 73 20 4d 61 63 42
0180 6f 6f 6b 20 41 69 72 c1 0c 00 21 80 01 c1 45 00
0190 10 80 01 15 41 6b 61 73 68 e2 80 99 73 20 4d 61
01a0 63 42 6f 6f 6b 20 41 69 72 c1 0c 00 21 80 01 c1
01b0 69 00 10 80 01 15 4d 61 64 64 79 e2 80 99 73 20
01c0 4d 61 63 42 6f 6f 6b 20 4e 65 6f c1 0c 00 21 80
01d0 01 c1 8b 00 10 80 01 19 4d 61 64 64 79 e2 80 99
01e0 73 20 4d 61 63 42 6f 6f 6b 20 4e 65 6f 20 28 32
01f0 29 c1 0c 00 21 80 01 c1 ad 00 10 80 01
Specifies if this is an individual (unicast) or group (broadcast/multicast) address (eth.dst.ig), 1 bit
Packets: 122691 · Displayed: 74215 (60.5%) · Dropped: 0 (0.0%)
Profile: Default


Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Length Info
93306 90.723865800 10.250.0.82 224.0.0.251 MDNS 509 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
93307 90.723867200 fe80::441:4fe:a7e5:… ff02::fb MDNS 529 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
Ethernet II, Src: 26:38:b2:38:98:b1 (26:38:b2:38:98:b1), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
Destination: IPv4mcast_fb (01:00:5e:00:00:fb)
.... ..0. .... = LG bit: Globally unique address (factory default)
....1 .... = IG bit: Group address (multicast/broadcast)
Source: 26:38:b2:38:98:b1 (26:38:b2:38:98:b1)
.... ..1. .... = LG bit: Locally administered address (this is NOT the factory de
....0 .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 2916]
Internet Protocol Version 4, Src: 10.250.0.82, Dst: 224.0.0.251
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
Total Length: 495
Identification: 0x96b7 (38583)
000. .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 255
Protocol: UDP (17)
Header Checksum: 0x363f [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.250.0.82
Destination Address: 224.0.0.251
[Stream index: 1333]
User Datagram Protocol, Src Port: 5353, Dst Port: 5353
Source Port: 5353
Destination Port: 5353
Length: 475
Checksum: 0xbd59 [unverified]
[Checksum Status: Unverified]
[Stream index: 2545]
[Stream Packet Number: 51]
[Timestamps]
0000 01 00 5e 00 00 fb 26 38 b2 38 98 b1 08 00 45 c0
0010 01 ef 96 b7 00 00 ff 11 36 3f 0a fa 00 52 e0 00
0020 00 fb 14 e9 14 e9 01 db bd 59 00 00 00 00 00 1b
0030 00 00 00 00 00 00 02 6c 62 07 5f 64 6e 73 2d 73
0040 64 04 5f 75 64 70 05 6c 6f 63 61 6c 00 00 0c 80
0050 01 08 5f 61 69 72 70 6f 72 74 04 5f 74 63 70 c0
0060 1c 00 0c 80 01 06 5f 75 73 63 61 6e c0 30 00 0c
0070 80 01 04 5f 70 74 70 c0 30 00 0c 80 01 0f 5f 70
0080 64 6c 2d 64 61 74 61 73 74 72 65 61 6d c0 30 00
0090 0c 80 01 04 5f 69 70 70 c0 30 00 0c 80 01 08 5f
00a0 73 63 61 6e 6e 65 72 c0 30 00 0c 80 01 08 5f 70
00b0 72 69 6e 74 65 72 c0 30 00 0c 80 01 07 5f 69 70
00c0 70 75 73 62 c0 30 00 0c 80 01 05 5f 69 70 73 73
00d0 c0 30 00 0c 80 01 07 5f 75 73 63 61 6e 73 c0 30
00e0 00 0c 80 01 05 5f 64 61 70 c0 30 00 0c 80 01
00f0 0d 5f 74 6f 75 63 68 2d 72 65 6d 6f 74 65 c0 30
0100 00 0c 80 01 08 5f 61 69 72 70 6c 61 79 c0 30 00
0110 0c 80 01 05 5f 72 61 6f 70 c0 30 00 0c 80 01 16
0120 43 68 61 72 76 69 e2 80 99 73 20 4d 61 63 42 6f
0130 6f 6b 20 50 72 6f 0e 5f 61 70 70 53 76 63 50 72
0140 65 50 61 69 72 c0 30 00 21 80 01 c0 f5 00 10 80
0150 01 11 53 52 49 56 41 52 53 48 20 50 41 4c 41 44
0160 55 47 55 c1 0c 00 21 80 01 c1 27 00 10 80 01 17
0170 4e 69 68 69 74 68 61 e2 80 99 73 20 4d 61 63 42
0180 6f 6f 6b 20 41 69 72 c1 0c 00 21 80 01 c1 45 00
0190 10 80 01 15 41 6b 61 73 68 e2 80 99 73 20 4d 61
01a0 63 42 6f 6f 6b 20 41 69 72 c1 0c 00 21 80 01 c1
01b0 69 00 10 80 01 15 4d 61 64 64 79 e2 80 99 73 20
01c0 4d 61 63 42 6f 6f 6b 20 4e 65 6f c1 0c 00 21 80
01d0 01 c1 8b 00 10 80 01 19 4d 61 64 64 79 e2 80 99
01e0 73 20 4d 61 63 42 6f 6f 6b 20 4e 65 6f 20 28 32
01f0 29 c1 0c 00 21 80 01 c1 ad 00 10 80 01


Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Length Info
93306 90.723865800 10.250.0.82 224.0.0.251 MDNS 509 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
93307 90.723867200 fe80::441:4fe:a7e5:… ff02::fb MDNS 529 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
User Datagram Protocol, Src Port: 5353, Dst Port: 5353
Source Port: 5353
Destination Port: 5353
Length: 475
Checksum: 0xbd59 [unverified]
[Checksum Status: Unverified]
[Stream index: 2545]
[Stream Packet Number: 51]
[Timestamps]
UDP payload (467 bytes)
Multicast Domain Name System (query)
Transaction ID: 0x0000
Flags: 0x0000 Standard query
Questions: 27
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
lb._dns-sd._udp.local: type PTR, class IN, "QU" question
_airport._tcp.local: type PTR, class IN, "QU" question
Name: _airport._tcp.local
[Name Length: 19]
[Label Count: 3]
Type: PTR (12) (domain name PoinTeR)
.000 0000 0000 0001 = Class: IN (0x0001)
1... .... = "QU" question: True
_uscan._tcp.local: type PTR, class IN, "QU" question
_ptp._tcp.local: type PTR, class IN, "QU" question
_pdl-datastream._tcp.local: type PTR, class IN, "QU" question
_ipp._tcp.local: type PTR, class IN, "QU" question
_scanner._tcp.local: type PTR, class IN, "QU" question
_printer._tcp.local: type PTR, class IN, "QU" question
_ippusb._tcp.local: type PTR, class IN, "QU" question
Bytes 471-498: Name (dns.qry.name)
Packets: 122691 · Displayed: 74215 (60.5%) · Dropped: 0 (0.0%)
Profile: Default


Skill-1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Length Info
93306 90.723865800 10.250.0.82 224.0.0.251 MDNS 509 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
93307 90.723867200 fe80::441:4fe:a7e5:… ff02::fb MDNS 529 Standard query 0x0000 PTR lb._dns-sd._udp.local, "QU" question PTR _airport._tcp.local, "QU" question PTR _uscan._tcp.loca
Multicast Domain Name System (query)
Transaction ID: 0x0000
Flags: 0x0000 Standard query
Questions: 27
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
lb._dns-sd._udp.local: type PTR, class IN, "QU" question
_airport._tcp.local: type PTR, class IN, "QU" question
Name: _airport._tcp.local
[Name Length: 19]
[Label Count: 3]
Type: PTR (12) (domain name PoinTeR)
.000 0000 0000 0001 = Class: IN (0x0001)
1... .... = "QU" question: True
_uscan._tcp.local: type PTR, class IN, "QU" question
_ptp._tcp.local: type PTR, class IN, "QU" question
_pdl-datastream._tcp.local: type PTR, class IN, "QU" question
_ipp._tcp.local: type PTR, class IN, "QU" question
_scanner._tcp.local: type PTR, class IN, "QU" question
_printer._tcp.local: type PTR, class IN, "QU" question
_ippusb._tcp.local: type PTR, class IN, "QU" question
_ipps._tcp.local: type PTR, class IN, "QU" question
_uscans._tcp.local: type PTR, class IN, "QU" question
_daap._tcp.local: type PTR, class IN, "QU" question
_touch-remote._tcp.local: type PTR, class IN, "QU" question
_airplay._tcp.local: type PTR, class IN, "QU" question
_raop._tcp.local: type PTR, class IN, "QU" question
Charvi's MacBook Pro._appSvcPrePair._tcp.local: type SRV, class IN, "QU" question
Charvi's MacBook Pro._appSvcPrePair._tcp.local: type TXT, class IN, "QU" question
SRIVARSH PALADUGU._appSvcPrePair._tcp.local: type SRV, class IN, "QU" question
SRIVARSH PALADUGU._appSvcPrePair._tcp.local: type TXT, class IN, "QU" question
Query Name (dns.qry.name), 16 bytes
Packets: 122691 · Displayed: 74215 (60.5%) · Dropped: 0 (0.0%)
Profile: Default



Record the following values:

Parameter	Where to find it
Frame length	Frame
Ethernet Header	Ethernet II
Source MAC Address	Ethernet II
Destination MAC Address	Ethernet II
IPv4 Header Length	Internet Protocol Version 4
IPv4 Total Length	Internet Protocol Version 4
TCP Header Length	Transmission Control Protocol
TCP Segment Length	Transmission Control Protocol

Repeat the process for one UDP packet. Observe the UDP header and its length field.

Step 6: Calculate header overhead

6.1 TCP Header Overhead Calculation

Based on the TCP packet captured in Wireshark:

Parameter	Observed value
Ethernet Header	14 bytes
IPv4 Header	20 bytes
TCP Header	20 bytes
TCP Payload	206 bytes
IPv4 Total Length	246 bytes
Frame Length	260 bytes

Total Frame Length:

14 + 20 + 20 + 206 = 260 bytes

Total Header Overhead:

14 + 20 + 20 = 54 bytes

Payload Efficiency:

$$\frac{206}{260} \times 100 = 79.23\%$$

Result:
The captured TCP packet has a total frame length of 260 bytes. Out of this, 54 bytes are used for protocol headers and 206 bytes carry TCP payload. The payload efficiency is approximately 79.23%.

6.2 UDP Header Overhead Calculation

Based on your UDP DNS response packet captured in Wireshark:

Parameter	**Observed value**
Ethernet Header including VLAN tag	18 bytes
IPv4 Header	20 bytes
UDP Header	8 bytes
UDP Payload	124 bytes
UDP Length	132 bytes
IPv4 Total Length	152 bytes
Frame Length	170 bytes

Total Frame Length:
18+20+8+124 = 170 bytes
Total Header Overhead:
18+20+8 = 46 bytes
Payload Efficiency:

$$\frac{124}{170} \times 100 = 72.94\%$$

Result:
The captured UDP packet has a total frame length of 170 bytes. Out of this, 46 bytes are used for protocol headers, including the VLAN tag, and 124 bytes carry UDP payload. The payload efficiency is approximately 72.94%.
Note: The UDP screenshot shows an 802.1Q VLAN tag, so its Ethernet header is 18 bytes rather than 14 bytes. The two captured frames' lengths match their Ethernet header plus IPv4 total length, so no additional FCS bytes are included in these calculations.

6.3 Comparison of TCP and UDP

Parameter	TCP	UDP
Frame Length	260 bytes	170 bytes
Header Overhead	54 bytes	46 bytes
Payload Length	206 bytes	124 bytes
Payload Efficiency	79.23%	72.94%

Observation:

TCP and UDP use different transport-layer headers. TCP has a 20-byte header in the selected packet, whereas UDP has an 8-byte header. The captured packets also have different Ethernet header sizes because the UDP packet contains a VLAN tag. Payload efficiency depends on both the packet size and the amount of application data carried.

Step 7: Transport Segment/Datagram Length

Record values from your own Wireshark capture.

Parameter	TCP Packet	UDP Packet
Frame Length	260 bytes	170 bytes
Ethernet Header Length	14 bytes	18 bytes
IP Header Length	20 bytes	20 bytes
Transport Header Length	20 bytes	8 bytes
IP Total Length	246 bytes	152 bytes
Transport Segment/Datagram Length	226 bytes	132 bytes
Payload Length	206 bytes	124 bytes

Important explanation:

- TCP segment length = 20-byte TCP header + 206-byte payload = 226 bytes.
- UDP datagram length = 132 bytes, as directly displayed in Wireshark.
- UDP payload = 132 − 8 = 124 bytes.
- UDP Ethernet header includes the 4-byte VLAN tag.

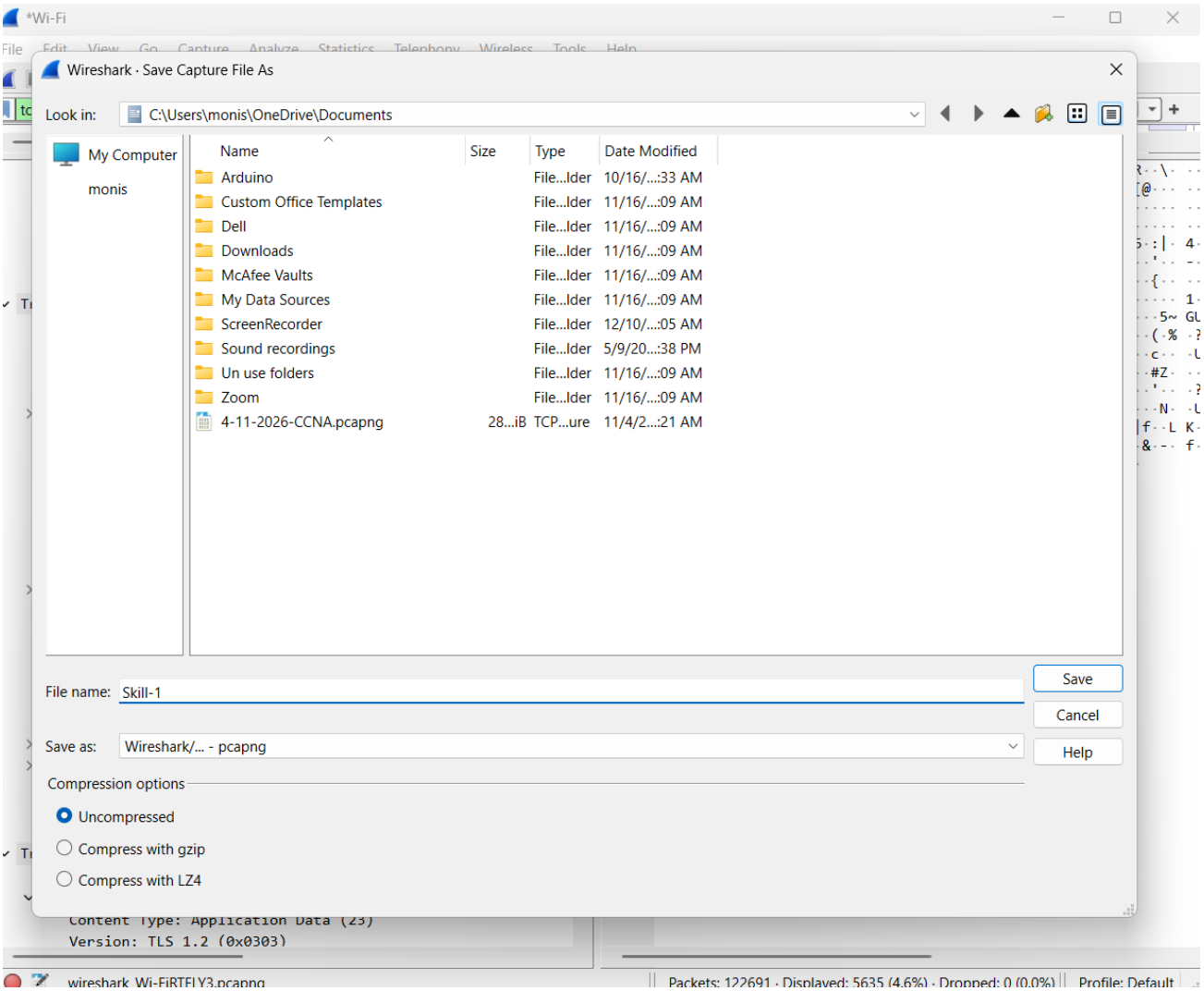
Observation:

- The TCP and UDP packets were successfully captured and analyzed using Wireshark. The Ethernet, IPv4, TCP and UDP headers were inspected, and their respective lengths were recorded.
- The captured TCP packet had a frame length of 260 bytes, with 54 bytes of header overhead and 206 bytes of payload. Its payload efficiency was calculated as 79.23%.
- The captured UDP packet had a frame length of 170 bytes, with 46 bytes of header overhead and 124 bytes of payload. Its payload efficiency was calculated as 72.94%.
- The observations demonstrate how protocol headers and VLAN tagging contribute to packet overhead and affect network efficiency.

Step 8: Save the capture

1. Click File in the menu.
2. Select Save As.
3. Choose the folder where you want to save the file.
4. Enter a filename.
5. Keep the file format as pcapng.
6. Click Save.

For example: Skill-1.pcapng



How to open a saved capture file

1. Open Wireshark.

2. Click File → Open.
3. Select your .pcapng or .pcap file.

4. Click Open.

5. Enter tcp.len > 0 in the display filter bar to view TCP packets carrying data.

